# ddelay

June 6, 2026

| | |
|---|---|
| **compile_options** | -lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single -ftz 0 |
| **delays.lib/name** | Faust Delay Library |
| **delays.lib/version** | 1.2.0 |
| **filename** | ddelay.dsp |
| **maths.lib/author** | GRAME |
| **maths.lib/copyright** | GRAME |
| **maths.lib/license** | LGPL with exception |
| **maths.lib/name** | Faust Math Library |
| **maths.lib/version** | 2.9.0 |
| **name** | ddelay |
| **platform.lib/name** | Generic Platform Library |
| **platform.lib/version** | 1.3.0 |
| **seam.lib/author** | Giuseppe Silvi, Davide Tedesco |
| **seam.lib/license** | CC3 |
| **seam.lib/name** | Faust SEAM main lib |
| **seam.lib/version** | 0.2 |
| **seam.math.lib/author** | Giuseppe Silvi |
| **seam.math.lib/license** | CC3 |
| **seam.math.lib/name** | SEAM Math - Library |
| **seam.math.lib/version** | 0.1 |

This document provides a mathematical description of the Faust program text stored in the `ddelay.dsp` file. See the notice in Section 3 (page 2) for details.

## 1 Mathematical definition of `process`

The *ddelay* program evaluates the signal transformer denoted by `process`, which is mathematically defined as follows:

1. Output signals $y_i$ for $i \in [1, 4]$ such that

$$y_1(t) = x_1(t - p_1(t))$$
$$y_2(t) = x_2(t - p_1(t))$$
$$y_3(t) = x_3(t - p_1(t))$$
$$y_4(t) = x_4(t - p_1(t))$$

2. Input signals $x_i$ for $i \in [1, 4]$

3. User-interface input signal $u_{s1}$ such that

$$\text{"Distance"} \quad (\text{m}) \quad u_{s1}(t) \in [\,0, 30\,] \quad (\text{default value} = 0)$$

4. Intermediate signal $p_1$ such that

$$p_1(t) = \min\left(32768, \max\left(0, \text{int}\left(k_1 \cdot u_{s1}(t)\right)\right)\right)$$

5. Constant $k_1$ such that

$$k_1 = 0.00301750150875075 \cdot \min\left(192000, \max\left(1, f_S\right)\right)$$

# 2 Block diagram of `process`

The block diagram of `process` is shown on Figure 1 (page 3).

# 3 Notice

- This document was generated using Faust version 2.85.5 on June 06, 2026.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the $f_S$ sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal $s$ in $S$ is a function mapping[1] times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from $S^n$ to $S^m$, where $n, m \in \mathbb{N}$. See the Faust manual for additional information (http://faust.grame.fr).
- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.

[1] Faust assumes that $\forall\, s \in S, \forall\, t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

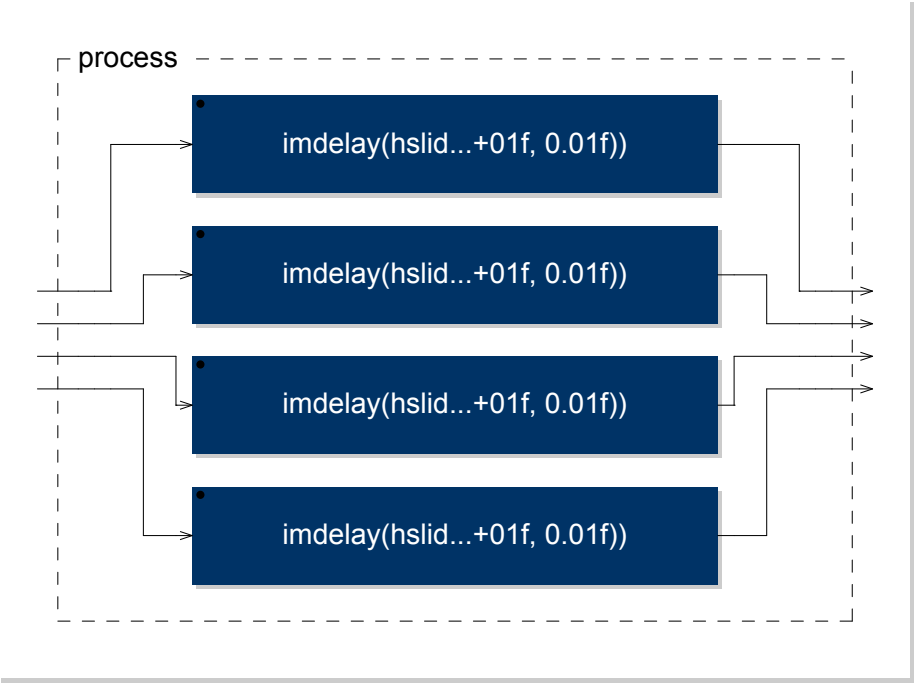


Figure 1: Block diagram of `process`

- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E; each graph is put in a box labeled by I. Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- $\forall\, x \in \mathbb{R}$,

$$\mathrm{int}(x) = \begin{cases} \lfloor x \rfloor & \text{if } x > 0 \\ \lceil x \rceil & \text{if } x < 0 \\ 0 & \text{if } x = 0 \end{cases} .$$

- The `ddelay-mdoc/` directory may also include the following subdirectories:
  - `cpp/` for Faust compiled code;
  - `pdf/` which contains this document;
  - `src/` for all Faust sources used (even libraries);
  - `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (http://www.w3.org/Graphics/SVG/);
  - `tex/` for the LaTeX source of this document.

# 4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

Listing 1: `ddelay.dsp`

```
import("seam.lib");
// Canonical DSP: sma.imdelay in seam.math.lib (interior distance delay).
// The plugin runs four parallel channels (quad speaker alignment).
distance = hslider("Distance [unit:m]", 0, 0, 30, 0.01);
process = par(i, 4, sma.imdelay(distance));
```

Listing 2: `seam.lib`

```
declare name "Faust SEAM main lib";
declare version "0.2";
declare author "Giuseppe Silvi";
declare author "Davide Tedesco";
declare license "CC3";
// calling standard faust libraries
import("stdfaust.lib");
// ### standard faust lib extensions
san = library("seam.analyzers.lib");
sba = library("seam.basic.lib");
sma = library("seam.math.lib");
sfi = library("seam.filters.lib");
sre = library("seam.reverbs.lib");
sno = library("seam.noises.lib");
// ### general Ambisonics theory
sam = library("seam.ambisonics.lib");
sdw = library("seam.dwt.lib");
// ### stereophony and perception
sst = library("seam.stereophony.lib");
// shr = library("seam.hrtf.lib");
// ### CSound and MaxMsp object cloning
scs = library("seam.csound.lib");
scy = library("seam.cyclone.lib");
// ### author specific literature
// sds = library("seam.discipio.lib");
smg = library("seam.gerzon.lib");
// sln = library("seam.nono.lib");
// sgn = library("seam.nottoli.lib");
sjm = library("seam.moorer.lib");
scr = library("seam.roads.lib");
sms = library("seam.schroeder.lib");
sfv = library("seam.freeverb.lib");

sff = library("seam.ffunctions.lib");

// ### live electronics stuff
// sgu = library("seam.gui.lib");
// shw = library("seam.hardware.lib");
// smx = library("seam.mixer.lib");

// ### instrument specific literature
// svc = library("seam.vcs3.lib");
```

Listing 3: `stdfaust.lib`

```
//################################## stdfaust.lib ##########################################
// The purpose of this library is to give access to all the Faust standard libraries
// through a series of environments.
//##########################################################################################

aa = library("aanl.lib");
sf = library("all.lib");
an = library("analyzers.lib");
ba = library("basics.lib");
co = library("compressors.lib");
db = library("debug.lib");
de = library("delays.lib");
dm = library("demos.lib");
dx = library("dx7.lib");
en = library("envelopes.lib");
fd = library("fds.lib");
fi = library("filters.lib");
ho = library("hoa.lib");
hy = library("hysteresis.lib");
it = library("interpolators.lib");
la = library("linearalgebra.lib");
ma = library("maths.lib");
mi = library("mi.lib");
mo = library("motion.lib");
ef = library("misceffects.lib");
no = library("noises.lib");
os = library("oscillators.lib");
pf = library("phaflangers.lib");
pl = library("platform.lib");
pm = library("physmodels.lib");
qu = library("quantizers.lib");
rm = library("reducemaps.lib");
re = library("reverbs.lib");
ro = library("routes.lib");
sp = library("spats.lib");
si = library("signals.lib");
so = library("soundfiles.lib");
sy = library("synths.lib");
ve = library("vaeffects.lib");
vl = library("version.lib");
wa = library("webaudio.lib");
wd = library("wdmodels.lib");
```

Listing 4: `seam.math.lib`

```
declare name "SEAM Math - Library";
declare version "0.1";
declare author "Giuseppe Silvi";
declare license "CC3";
//
import("seam.lib");
//============================================================ MATH FUNCTIONS ===
//=============================================================================
sma = library("seam.math.lib");
//------------------------------------------------------------------------ PI ---
//-------------------------------------------------- QUAD PRECISION 81 DECIMALS ---
PIq = 3.14159265358979323846264338327950288419716939937510582097494459230781640;
// process = PIq;
PIc = atan(1)*4;
// process = PIc;
twoPI = 2*ma.PI;
// process = twoPI;
```

```
//------------------------------------------------------------------- OMEGA ---
omega(fc) = fc*twoPI/ma.SR;
// process = omega(24000);
//---------------------------------------------------- BILINEAR TRANSFORM ---
w(fc) = tan(ma.PI*fc/ma.SR/2);
// process = w(24000);
//------------------------------------------------------------ SIN^2 - COS^2 ---
cosq(x) = cos(x)*cos(x);
// process = cosq(ma.PI);
sinq(x) = sin(x)*sin(x);
// process = sinq(3/2*ma.PI);
//=================================================== POLAR | CARTESIAN ===
//=========================================================================
// https://en.wikipedia.org/wiki/Inverse_trigonometric_functions
// https://en.wikipedia.org/wiki/Spherical_coordinate_system
// https://en.wikipedia.org/wiki/Atan2
//---------------------------------------------------- DEGREES TO RADIANS ---
d2r(d) = d*(ma.PI/180);
// process = d2r(0);
//---------------------------------------------------- RADIANS TO DEGREES ---
r2d(r) = r/ma.PI*180;
// process = r2d(0);
//------------------------------------------------------- AED TO CARTESIAN ---
aed2xyz(a,e,d) = cx(a,e,d), cy(a,e,d), cz(a,e,d)
with{
  cx(a,e,d) = d*(cos(a)*cos(e));
  cy(a,e,d) = d*(sin(a)*cos(e));
  cz(a,e,d) = d*(sin(e));
};
// process = aed2xyz(0,0,1);
//------------------------------------------------------- CARTESIAN TO AED ---
xyz2aed(x,y,z) = a(x,y),e(x,y,z),d(x,y,z)
with{
  a(x,y)  = atan2(y,x);
  e(x,y,z) = atan2(sqrt(x^2+y^2),z);
  d(x,y,z) = sqrt(x^2+y^2+z^2);
};
// process = xyz2aed(1,1,1) : r2d, r2d, _;
//------------------------------------------------------------------------ PHI ---
phi(x) = x*(1+(sqrt(5)))/2;
// process = phi(1);
//
// reciprocal o the golden ratio
rphi(x) = x*(sqrt(5)-1)/2;
// process = rphi(1);
//
// phi progression
srphi(0,x) = x;
srphi(i,x) = rphi(x) : srphi(i-1);
// process = par(i,16,srphi(i,sba.nyq));
//------------------------------------------------------------- FACTORIALS ---
factorial(0) = 1;
factorial(i) = i * factorial(i-1);
// permutazioni possibili senza ricorsioni
permutation(k,n) = factorial(k)/factorial(n);
//---------------------------------------------------------- SPEED OF SOUND ---
esos = 344; // exterior
isos = 331.4; // interior
//---------------------------------------------------------- METERS TO SAMPLES ---
emt2samp(mt) = int(mt*ma.SR/esos);
imt2samp(mt) = int(mt*ma.SR/isos);
//------------------------------------------------ INTERIOR DISTANCE DELAY ---
// Pure integer-sample delay for a distance in metres, using the interior
// speed of sound (isos). Max line 1<<15 samples (~0.68 s at 48 kHz, ~226 m).
// SEAM-LTM DDELAY plugin (per-channel; the plugin runs four in parallel).
imdelay(mt) = de.delay(1 << 15, imt2samp(mt));
```

Listing 5: `delays.lib`

```
//################################## delays.lib ########################################
// Delays library. Its official prefix is `de`.
//
// This library provides reusable building blocks for delay-based processing:
// single and multi-tap delays, fractional delays and utilities for echo and spatial effects.
//
// The Delays library is organized into 4 sections:
//
// * [Basic Delay Functions](#basic-delay-functions)
// * [Lagrange Interpolation](#lagrange-interpolation)
// * [Thiran Allpass Interpolation](#thiran-allpass-interpolation)
// * [Others](#others)
//
// #### References
//
// * <https://github.com/grame-cncm/faustlibraries/blob/master/delays.lib>
//######################################################################################

/************************************************************************
************************************************************************
FAUST library file
Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
--------------------------------------------------------------------
This program is free software; you can redistribute it and/or modify
it under the terms of the GNU Lesser General Public License as
published by the Free Software Foundation; either version 2.1 of the
License, or (at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public
License along with the GNU C Library; if not, write to the Free
Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
02111-1307 USA.

EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
larger FAUST program which directly or indirectly imports this library
file and still distribute the compiled code generated by the FAUST
compiler, or a modified version of this compiled code, under your own
copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
grants you the right to freely choose the license for the resulting
compiled code. In particular the resulting compiled code has no obligation
to be LGPL or GPL. For example you are free to choose a commercial or
closed source license or any other license if you decide so.
************************************************************************
************************************************************************/

ma = library("maths.lib");
ba = library("basics.lib");
si = library("signals.lib");
fi = library("filters.lib");

declare name "Faust Delay Library";
declare version "1.2.0";

//================================Basic Delay Functions================================
//=====================================================================================

//-------`(de.)delay`----------
// Simple `d` samples delay where `n` is the maximum delay length as a number of
// samples. Unlike the `@` delay operator, here the delay signal `d` is explicitly
// bounded to the interval [0..n]. The consequence is that delay will compile even
// if the interval of d can't be computed by the compiler.
```

```
// `delay` is a standard Faust function.
//
// #### Usage
//
// ```
// _ : delay(n,d) : _
// ```
//
// Where:
//
// * `n`: the max delay length in samples
// * `d`: the delay length in samples (integer)

// #### Test
// ```
// de = library("delays.lib");
// os = library("oscillators.lib");
// delay_test = os.osc(440) : de.delay(44100, 22050);
// ```
//
// TODO: add MBH np2
//------------------------------
delay(n,d,x) = x @ min(n, max(0,d));


//-------`(de.)fdelay`----------
// Simple `d` samples fractional delay based on 2 interpolated delay lines where `n` is
// the maximum delay length as a number of samples.
//
// `fdelay` is a standard Faust function.
//
// #### Usage
//
// ```
// _ : fdelay(n,d) : _
// ```
//
// Where:
//
// * `n`: the max delay length in samples
// * `d`: the delay length in samples (float)
//
// #### Test
// ```
// de = library("delays.lib");
// os = library("oscillators.lib");
// fdelay_test = os.osc(440) : de.fdelay(44100, 22050.5);
// ```
//------------------------------
fdelay(n,d,x) = delay(n+1,int(d),x)*(1 - ma.frac(d)) + delay(n+1,int(d)+1,x)*ma.frac(d);


//--------------------------`(de.)sdelay`-----------------------------
// s(mooth)delay: a mono delay that doesn't click and doesn't
// transpose when the delay time is changed.
//
// #### Usage
//
// ```
// _ : sdelay(n,it,d) : _
// ```
//
// Where:
//
// * `n`: the max delay length in samples
// * `it`: interpolation time (in samples), for example 1024
// * `d`: the delay length in samples (float)
//
```

```
// #### Test
// ```
// de = library("delays.lib");
// os = library("oscillators.lib");
// sdelay_test = os.osc(440) : de.sdelay(44100, 1024, 22050.5);
// ```
//------------------------------
sdelay(n, it, d) = ctrl(it,d),_ : ddi(n)
with {
    // ddi(n,i,d0,d1)
    // DDI (Double Delay with Interpolation) : the input signal is sent to two
    // delay lines. The outputs of these delay lines are crossfaded with
    // an interpolation stage. By acting on this interpolation value one
    // can move smoothly from one delay to another. When <i> is 0 we can
    // freely change the delay time <d1> of line 1, when it is 1 we can freely change
    // the delay time <d0> of line 0.
    //
    // <n> = maximal delay in samples
    // <i> = interpolation value between 0 and 1 used to crossfade the outputs of the
    //       two delay lines (0.0: first delay line, 1.0: second delay line)
    // <d0> = delay time of delay line 0 in samples between 0 and <N>-1
    // <d1> = delay time of delay line 1 in samples between 0 and <N>-1
    // < > = the input signal we want to delay
    ddi(n, i, d0, d1) = _ <: delay(n,d0), delay(n,d1) : si.interpolate(i);

    // ctrl(it,d)
    // Control logic for a Double Delay with Interpolation according to two
    //
    // USAGE : ctrl(it,d)
    // where :
    // <it> an interpolation time (in samples, for example 256)
    // <d> a delay time (in samples)
    //
    // ctrl produces 3 outputs : an interpolation value <i> and two delay
    // times <d0> and <d1>. These signals are used to control a ddi (Double Delay with
       Interpolation).
    // The principle is to detect changes in the input delay time d, then to
    // change the delay time of the delay line currently unused and then by a
    // smooth crossfade to remove the first delay line and activate the second one.
    //
    // The control logic has an internal state controlled by 4 elements
    // <v> : the interpolation variation (0, 1/it, -1/it)
    // <i> : the interpolation value (between 0 and 1)
    // <d0>: the delay time of line 0
    // <d1>: the delay time of line 1
    //
    // Please note that the last stage (!,_,_,_) cut <v> because it is only
    // used internally.
    ctrl(it, d) = \(v,ip,d0,d1).((nv, nip, nd0, nd1)
    with {
        // interpolation variation
        nv = ba.if (v!=0.0,                 // if variation we are interpolating
            ba.if ((ip>0.0) & (ip<1.0), v, 0), // should we continue or not ?
            ba.if ((ip==0.0) & (d!=d0), 1.0/it, // if true xfade from d10 to d11
            ba.if ((ip==1.0) & (d!=d1), -1.0/it, // if true xfade from d11 to d10
            0)));                           // nothing to change
        // interpolation value
        nip = ip+nv : min(1.0) : max(0.0);

        // update delay time of line 0 if needed
        nd0 = ba.if ((ip >= 1.0) & (d1!=d), d, d0);

        // update delay time of line 1 if needed
        nd1 = ba.if ((ip <= 0.0) & (d0!=d), d, d1);
    }) ~ (_,_,_,_) : (!,_,_,_);
};
```

```
//------------------`(de.)prime_power_delays`----------------
// Prime Power Delay Line Lengths.
//
// #### Usage
//
// ```
// si.bus(N) : prime_power_delays(N,pathmin,pathmax) : si.bus(N);
// ```
//
// Where:
//
// * `N`: positive integer up to 16 (for higher powers of 2, extend 'primes' array below)
// * `pathmin`: minimum acoustic ray length in the reverberator (in meters)
// * `pathmax`: maximum acoustic ray length (meters) - think "room size"
//
// #### Test
// ```
// de = library("delays.lib");
// prime_power_delays_test = de.prime_power_delays(4, 1, 10);
// ```
//
// #### References
//
// * <https://ccrma.stanford.edu/~jos/pasp/Prime_Power_Delay_Line.html>
//-----------------------------------------------------------
declare prime_power_delays author "Julius O. Smith III";

prime_power_delays(N,pathmin,pathmax) = par(i,N,delayvals(i)) with {
  Np = 16;
  primes = 2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53;
  prime(n) = primes : ba.selector(n,Np); // math.lib

  // Prime Power Bounds [matlab: floor(log(maxdel)./log(primes(53)))]
  maxdel = 8192; // more than 63 meters at 44100 samples/sec & 343 m/s
  ppbs = 13,8,5,4, 3,3,3,3, 2,2,2,2, 2,2,2,2; // 8192 is enough for all
  ppb(i) = ba.take(i+1,ppbs);

  // Approximate desired delay-line lengths using powers of distinct primes:
  c = 343; // soundspeed in m/s at 20 degrees C for dry air
  dmin = ma.SR*pathmin/c;
  dmax = ma.SR*pathmax/c;
  dl(i) = dmin * (dmax/dmin)^(i/float(N-1)); // desired delay in samples
  ppwr(i) = floor(0.5+log(dl(i))/log(prime(i))); // best prime power
  delayvals(i) = prime(i)^ppwr(i); // each delay a power of a distinct prime
};


//==============================Lagrange Interpolation==================================
//======================================================================================

//----------------------`(de.)fdelaylti` and `(de.)fdelayltv`-------------------------
// Fractional delay line using Lagrange interpolation.
//
// #### Usage
//
// ```
// _ : fdelaylt[i|v](N, n, d) : _
// ```
//
// Where:
//
// * `N=1,2,3,...` is the order of the Lagrange interpolation polynomial (constant numerical
    expression)
// * `n`: the max delay length in samples
// * `d`: the delay length in samples
//
// `fdelaylti` is most efficient, but designed for constant/slowly-varying delay.
// `fdelayltv` is more expensive and more robust when the delay varies rapidly.
```

```
//
// Note: the requested delay should not be less than `(N-1)/2`.
//
// #### Test
// ```
// de = library("delays.lib");
// os = library("oscillators.lib");
// fdelaylti_test = os.osc(440) : de.fdelaylti(3, 44100, 22050.5);
// ```
//
// #### References
//
// * <https://ccrma.stanford.edu/~jos/pasp/Lagrange_Interpolation.html>
// * [Fixed-delay case](https://ccrma.stanford.edu/~jos/Interpolation/
     Efficient_Time_Invariant_Lagrange_Interpolation.html)
// * [Variable-delay case](https://ccrma.stanford.edu/~jos/Interpolation/
     Time_Varying_Lagrange_Interpolation.html)
// * Timo I. Laakso et al., "Splitting the Unit Delay - Tools for Fractional Delay Filter
     Design", IEEE Signal Processing Magazine, vol. 13, no. 1, pp. 30-60, Jan 1996.
// * Philippe Depalle and Stephan Tassart, "Fractional Delay Lines using Lagrange
     Interpolators", ICMC Proceedings, pp. 341-343, 1996.
//------------------------------------------------------------
declare fdelaylti author "Julius O. Smith III";

fdelaylti(N,n,d,x) = delay(n,id,x) <: seq(i,N,section(i)) : !,_
with {
  o = (N-1.00001)/2; // offset to ~center FIR interpolator
  dmo = d - o; // assumed nonnegative [d > (N-1)/2]
  id = int(dmo);
  fd = o + ma.frac(dmo);
  section(i,x,y) = (x-x') * c(i) <: _,+(y);
  c(i) = (i - fd)/(i+1);
};

declare fdelayltv author "Julius O. Smith III";

fdelayltv(N,n,d,x) = sum(i, N+1, delay(n,id+i,x) * h(N,fd,i))
with {
  o = (N-1.00001)/2; // ~center FIR interpolator
  dmo = d - o; // assumed nonnegative [d > (N-1)/2]
  id = int(dmo);
  fd = o + ma.frac(dmo);
  h(N,d,n) = facs1(N,d,n) * facs2(N,d,n);
  facs1(N,d,n) = select2(n,1,prod(k,max(1,n),select2(k<n,1,fac(d,n,k))));
  facs2(N,d,n) = select2(n<N,1,prod(l,max(1,N-n),fac(d,n,l+n+1)));
  fac(d,n,k) = (d-k)/((n-k)+(n==k));
  // Explicit formula for Lagrange interpolation coefficients:
  // h_d(n) = \prod_{\stackrel{k=0}{k\neq n}}^N \frac{d-k}{n-k}, n=0:N
};

//------------------`(de.)fdelay[N]`-----------------------------
// For convenience, `fdelay1`, `fdelay2`, `fdelay3`, `fdelay4`, `fdelay5`
// are also available where `N` is the order of the interpolation, built using `fdelayltv`.
//------------------------------------------------------------
declare fdelay1 author "Julius O. Smith III";
declare fdelay2 author "Julius O. Smith III";
declare fdelay3 author "Julius O. Smith III";
declare fdelay4 author "Julius O. Smith III";
declare fdelay5 author "Julius O. Smith III";

fdelay1 = fdelayltv(1);
fdelay2 = fdelayltv(2);
fdelay3 = fdelayltv(3);
fdelay4 = fdelayltv(4);
fdelay5 = fdelayltv(5);

//==============================Thiran Allpass Interpolation================================
// Thiran Allpass Interpolation.
```

```
//
// #### References
//
// * <https://ccrma.stanford.edu/~jos/pasp/Thiran_Allpass_Interpolators.html>
//================================================================================================

//----------------`(de.)fdelay[N]a`-------------
// Delay lines interpolated using Thiran allpass interpolation.
//
// #### Usage
//
// ```
// _ : fdelay[N]a(n, d) : _
// ```
//
// (exactly like `fdelay`)
//
// Where:
//
// * `N=1,2,3, or 4` is the order of the Thiran interpolation filter (constant numerical expression),
//   and the delay argument is at least `N-1/2`. First-order: `d` at least 0.5, second-order : `d` at least 1.5,
//   third-order: `d` at least 2.5, fourth-order: `d` at least 3.5.
// * `n`: the max delay length in samples
// * `d`: the delay length in samples
//
// #### Test
// ```
// de = library("delays.lib");
// os = library("oscillators.lib");
// fdelay2a_test = os.osc(440) : de.fdelay2a(44100, 22050.5);
// ```
//
// #### Note
//
// The interpolated delay should not be less than `N-1/2`.
// (The allpass delay ranges from `N-1/2` to `N+1/2`).
// This constraint can be alleviated by altering the code,
// but be aware that allpass filters approach zero delay
// by means of pole-zero cancellations.
//
// Delay arguments too small will produce an UNSTABLE allpass!
//
// Because allpass interpolation is recursive, it is not as robust
// as Lagrange interpolation under time-varying conditions
// (you may hear clicks when changing the delay rapidly).
//
//------------------------------------------------------------
declare fdelay1a author "Julius O. Smith III";

fdelay1a(n,d,x) = delay(n,id,x) : fi.tf1(eta,1,eta)
with {
  o = 0.49999; // offset to make life easy for allpass
  dmo = d - o; // assumed nonnegative
  id = int(dmo);
  fd = o + ma.frac(dmo);
  eta = (1-fd)/(1+fd); // allpass coefficient
};

declare fdelay2a author "Julius O. Smith III";
fdelay2a(n,d,x) = delay(n,id,x) : fi.tf2(a2,a1,1,a1,a2)
with {
  o = 1.49999;
  dmo = d - o; // delay range is [order-1/2, order+1/2]
  id = int(dmo);
  fd = o + ma.frac(dmo);
  a1o2 = (2-fd)/(1+fd); // share some terms (the compiler does this anyway)
```

```
  a1 = 2*a1o2;
  a2 = a1o2*(1-fd)/(2+fd);
};

declare fdelay3a author "Julius O. Smith III";
fdelay3a(n,d,x) = delay(n,id,x) : fi.iir((a3,a2,a1,1),(a1,a2,a3))
with {
  o = 2.49999;
  dmo = d - o;
  id = int(dmo);
  fd = o + ma.frac(dmo);
  a1o3 = (3-fd)/(1+fd);
  a2o3 = a1o3*(2-fd)/(2+fd);
  a1 = 3*a1o3;
  a2 = 3*a2o3;
  a3 = a2o3*(1-fd)/(3+fd);
};

declare fdelay4a author "Julius O. Smith III";
fdelay4a(n,d,x) = delay(n,id,x) : fi.iir((a4,a3,a2,a1,1),(a1,a2,a3,a4))
with {
  o = 3.49999;
  dmo = d - o;
  id = int(dmo);
  fd = o + ma.frac(dmo);
  a1o4 = (4-fd)/(1+fd);
  a2o6 = a1o4*(3-fd)/(2+fd);
  a3o4 = a2o6*(2-fd)/(3+fd);
  a1 = 4*a1o4;
  a2 = 6*a2o6;
  a3 = 4*a3o4;
  a4 = a3o4*(1-fd)/(4+fd);
};

//================================Others=============================================
//===================================================================================

//----------------`(de.)multiTapSincDelay`-------------
//
// Variable delay line using multi-tap sinc interpolation.
//
// This function implements a continuously variable delay line by superposing (2K+2) auxiliary delayed signals
// whose positions and gains are determined by a sinc-based interpolation method. It extends the traditional
// crossfade delay technique to significantly reduce spectral coloration artifacts, which are problematic in
// applications like Wave Field Synthesis (WFS) and auralization.
//
// Operation:
//
//  - If tau1 and tau2 are very close (|tau2 - tau1| 0), a simple fixed fractional delay is applied
//  - Otherwise, a variable delay is synthesized by:
//
//    - Computing (2K+2) taps symmetrically distributed around tau1 and tau2
//    - Applying sinc-based weighting to each tap, based on its offset from the target interpolated delay tau
//    - Summing all the weighted taps to produce the output
//
// Features:
//
//  - Smooth delay variation without introducing Doppler pitch shifts
//  - Significant reduction of comb-filter coloration compared to classical crossfading
//  - Switching between fixed and variable delay modes to ensure stability
//
// #### Usage
//
```

```
// ```
// _ : multiTapSincDelay(K, MaxDelay, tau1, tau2, alpha) : _
// ```
//
// Where:
//
// * `K (integer)`: number of auxiliary tap pairs (a constant numerical expression). Total number of taps = 2*K + 2
// * `MaxDelay`: maximum allowable delay in samples (buffer size)
// * `tau1`: initial delay in samples (can be fractional)
// * `tau2`: target delay in samples (can be fractional)
// * `alpha`: interpolation factor between tau1 and tau2 (in [0,1] with 0 = tau1, 1 = tau2)
//
// #### Test
// ```
// de = library("delays.lib");
// os = library("oscillators.lib");
// multiTapSincDelay_test = os.osc(440) : de.multiTapSincDelay(2, 4096, 1024.0, 1536.0, 0.5) ;
// ```
//
// #### References
//
// T. Carpentier, "Implementation of a continuously variable delay line by crossfading between several tap delays", 2024: <https://hal.science/hal-04646939>
//------------------------------------------------------------
multiTapSincDelay(K, MaxDelay, tau1, tau2, alpha, input) = output with {

    // Total number of taps used in the sum (2K + 2).
    numTaps = 2 * K + 2;

    // Normalized Sinc function: sinc(x) = sin(pi*x)/(pi*x)
    // Handles the case where x  0 to avoid division by zero and return 1.
    sinc(x) = ba.if(abs(x) < ma.EPSILON, 1, sin(ma.PI * x) / (ma.PI * x));

    // Computes the difference between two delays: delta = tau2 - tau1
    delta(t1, t2) = t2 - t1;

    // Computes the target interpolated delay: tau = (1-alpha)*tau1 + alpha*tau2
    // This is the center of the sinc function for the hk gains.
    tau(t1, t2, a) = (1 - a) * t1 + a * t2;

    // Computes the position (delay in samples) of the i-th tap (tk),
    // according to equation (17) of the article.
    tk(i, t1, t2) = ba.if(i <= K,
                      t1 - (K - i) * delta(t1, t2),
                      t2 + (i - K - 1) * delta(t1, t2));

    // Computes the gain (hk) of the i-th tap, according to equation (19) of the paper.
    // hk = sinc((tk - tau) / delta)
    hk(i, t1, t2, a) = sinc(offset) with {
        d     = delta(t1, t2);
        denom = ba.if(abs(d) < ma.EPSILON, 1, d);
        offset = (tk(i, t1, t2) - tau(t1, t2, a)) / denom;
    };

    // Computes the sum of delayed signals weighted by their gains.
    // This is the core of the variable multi-tap effect.
    tapSum(sig, t1, t2, a) =
        // Iterates from i = 0 to numTaps-1
        sum(i, numTaps,
            // For each tap 'i':
            // 1. Delays the signal 'sig' by tk(i) samples (fractional)
            //   using an internal delay line (fdelay).
            // 2. Multiplies the delayed signal by the gain hk(i).
            sig : fdelay(MaxDelay, tk(i, t1, t2)) * hk(i, t1, t2, a)
        );

```

```
    // Simple function to apply a fixed delay (potentially fractional).
    // Used when tau1 and tau2 are very close.
    fixedDelay(x, t) = x : fdelay(MaxDelay, t);

    // Determines if delays tau1 and tau2 are close enough to be
    // considered fixed (to avoid unstable calculations when delta  0).
    // isFixed equals 1 if delta  0, otherwise 0.
    isFixed = abs(delta(tau1, tau2)) < ma.EPSILON;

    // Computes the output corresponding to the fixed delay (used if isFixed=1).
    fd = fixedDelay(input, tau1);

    // Computes the output corresponding to the variable multi-tap delay (used if isFixed=0).
    mt = tapSum(input, tau1, tau2, alpha);

    // Selects the final output based on isFixed.
    output = ba.if(isFixed, fd, mt);
};


////////////////////////////////Deprecated Functions/////////////////////////////////////
// This section implements functions that used to be in music.lib but that are now
// considered as "deprecated".
//////////////////////////////////////////////////////////////////////////////////////////

delay1s(d) = delay(65536,d);
delay2s(d) = delay(131072,d);
delay5s(d) = delay(262144,d);
delay10s(d) = delay(524288,d);
delay21s(d) = delay(1048576,d);
delay43s(d) = delay(2097152,d);

fdelay1s(d) = fdelay(65536,d);
fdelay2s(d) = fdelay(131072,d);
fdelay5s(d) = fdelay(262144,d);
fdelay10s(d) = fdelay(524288,d);
fdelay21s(d) = fdelay(1048576,d);
fdelay43s(d) = fdelay(2097152,d);
```

Listing 6: `maths.lib`

```
//################################ maths.lib ########################################
// Maths library. Its official prefix is `ma`.
//
// This library provides mathematical functions and utilities for numerical
// computations in Faust. It includes trigonometric, exponential, logarithmic, and
// statistical functions, constants, and complex-number
// operations used throughout Faust DSP and control code.
//
// The Maths library is organized into 1 section:
//
// * [Functions Reference](#functions-reference)
//
// Some functions are implemented as Faust foreign functions of `math.h` functions
// that are not part of Faust's primitives. Defines also various constants and several
// utilities.
//
// #### References
//
// * <https://github.com/grame-cncm/faustlibraries/blob/master/maths.lib>
//###########################################################################################

// ## History
// * 06/13/2016 [RM] normalizing and integrating to new libraries
```

```
// * 07/08/2015 [YO] documentation comments
// * 20/06/2014 [SL] added FTZ function
// * 22/06/2013 [YO] added float|double|quad variants of some foreign functions
// * 28/06/2005 [YO] postfixed functions with 'f' to force float version instead of double
// * 28/06/2005 [YO] removed 'modf' because it requires a pointer as argument

/************************************************************************
************************************************************************
FAUST library file
Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
--------------------------------------------------------------------
This program is free software; you can redistribute it and/or modify
it under the terms of the GNU Lesser General Public License as
published by the Free Software Foundation; either version 2.1 of the
License, or (at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public
License along with the GNU C Library; if not, write to the Free
Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
02111-1307 USA.

EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
larger FAUST program which directly or indirectly imports this library
file and still distribute the compiled code generated by the FAUST
compiler, or a modified version of this compiled code, under your own
copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
grants you the right to freely choose the license for the resulting
compiled code. In particular the resulting compiled code has no obligation
to be LGPL or GPL. For example you are free to choose a commercial or
closed source license or any other license if you decide so.
************************************************************************
************************************************************************/

// This library contains platform specific constants
ba = library("basics.lib");
pl = library("platform.lib");
ma = library("maths.lib"); // for compatible copy/paste out of this file

declare name "Faust Math Library";
declare version "2.9.0";
declare author "GRAME";
declare copyright "GRAME";
declare license "LGPL with exception";

//============================Functions Reference=======================================
//========================================================================================


//---------------------------------`(ma.)SR`---------------------------------------
// Current sampling rate given at init time. Constant during program execution.
//
// #### Usage
//
// ```
// SR : _
// ```
//
// Where:
//
// * `SR`: initialization-time sampling rate constant
//
// #### Test
// ```
```

```
// ma = library("maths.lib");
// SR_test = ma.SR;
// ```
//-----------------------------------------------------------------------------
SR = pl.SR;

//----------------------------------`(ma.)T`---------------------------------------
// Current sample duration in seconds computed from the sampling rate given at init time.
     Constant during program execution.
//
// #### Usage
//
// ```
// T : _
// ```
//
// Where:
//
// * `T`: sample duration (`1/SR`) constant
//
// #### Test
// ```
// ma = library("maths.lib");
// T_test = ma.T;
// ```
//-----------------------------------------------------------------------------
T = 1.0 / SR;

//----------------------------------`(ma.)BS`--------------------------------------
// Current block-size. Can change during the execution at each block.
//
// #### Usage
//
// ```
// BS : _
// ```
//
// Where:
//
// * `BS`: current processing block size
//
// #### Test
// ```
// ma = library("maths.lib");
// BS_test = ma.BS;
// ```
//-----------------------------------------------------------------------------
BS = pl.BS;


//----------------------------------`(ma.)PI`--------------------------------------
// Constant PI in double precision.
//
// #### Usage
//
// ```
// PI : _
// ```
//
// Where:
//
// * `PI`: double-precision  constant
//
// #### Test
// ```
// ma = library("maths.lib");
// PI_test = ma.PI;
// ```
```

```
//-------------------------------------------------------------------------
PI = 3.14159265358979323846;


//----------------------------------`(ma.)deg2rad`----------------------------------
// Convert degrees to radians.
//
// #### Usage
//
// ```
// 45. : deg2rad
// ```
//
// Where:
//
// * input: angle in degrees to convert
//
// #### Test
// ```
// ma = library("maths.lib");
// deg2rad_test = 45.0 : ma.deg2rad;
// ```
//-------------------------------------------------------------------------
deg2rad = _ * PI / 180.;


//----------------------------------`(ma.)rad2deg`----------------------------------
// Convert radians to degrees.
//
// #### Usage
//
// ```
// ma.PI : rad2deg
// ```
//
// Where:
//
// * input: angle in radians to convert
//
// #### Test
// ```
// ma = library("maths.lib");
// rad2deg_test = ma.PI : ma.rad2deg;
// ```
//-------------------------------------------------------------------------
rad2deg = _ * 180. / PI;


//----------------------------------`(ma.)E`----------------------------------------
// Constant e in double precision.
//
// #### Usage
//
// ```
// E : _
// ```
//
// Where:
//
// * `E`: double-precision Euler's number constant
//
// #### Test
// ```
// ma = library("maths.lib");
// E_test = ma.E;
// ```
//-------------------------------------------------------------------------
E = 2.71828182845904523536;
```

```
227
228
229 //--------------------------------`(ma.)EPSILON`---------------------------------------
230 // Constant EPSILON available in simple/double/quad precision,
231 // as defined in the [floating-point standard](https://en.wikipedia.org/wiki/IEEE_754)
232 // and [machine epsilon](https://en.wikipedia.org/wiki/Machine_epsilon),
233 // that is smallest positive number such that `1.0 + EPSILON != 1.0`.
234 //
235 // #### Usage
236 //
237 // ```
238 // EPSILON : _
239 // ```
240 //
241 // Where:
242 //
243 // * `EPSILON`: machine epsilon constant for the current floating-point precision
244 //
245 // #### Test
246 // ```
247 // ma = library("maths.lib");
248 // EPSILON_test = ma.EPSILON;
249 // ```
250 //-------------------------------------------------------------------------------
251 singleprecision EPSILON = 1.192092896e-07;
252 doubleprecision EPSILON = 2.2204460492503131e-016;
253 quadprecision EPSILON = 1.08420217248550443400745 2e-019;
254 fixedpointprecision EPSILON = 2.2204460492503131e-016;
255
256
257 //--------------------------------`(ma.)MIN`---------------------------------------
258 // Constant MIN available in simple/double/quad precision (minimal positive value).
259 //
260 // #### Usage
261 //
262 // ```
263 // MIN : _
264 // ```
265 //
266 // Where:
267 //
268 // * `MIN`: minimal positive normalized value for the current precision
269 //
270 // #### Test
271 // ```
272 // ma = library("maths.lib");
273 // MIN_test = ma.MIN * 1e307;
274 // ```
275 //-------------------------------------------------------------------------------
276 singleprecision MIN = 1.175494351e-38;
277 doubleprecision MIN = 2.2250738585072014e-308;
278 quadprecision MIN = 2.2250738585072014e-308;
279 fixedpointprecision MIN = 2.2250738585072014e-308;
280
281
282 //--------------------------------`(ma.)MAX`-----------------------------
283 // Constant MAX available in simple/double/quad precision (maximal positive value).
284 //
285 // #### Usage
286 //
287 // ```
288 // MAX : _
289 // ```
290 //
291 // Where:
292 //
293 // * `MAX`: maximal finite value for the current precision
294 //
```

```
// #### Test
// ```
// ma = library("maths.lib");
// MAX_test = ma.MAX;
// ```
//-------------------------------------------------------------------------
singleprecision MAX = 3.402823466e+38;
doubleprecision MAX = 1.7976931348623158e+308;
quadprecision MAX = 1.7976931348623158e+308;
fixedpointprecision MAX = 1.7976931348623158e+308;

// Obsolete, kept for compatibility reasons
INFINITY = MAX;

//----------------------------------`(ma.)FTZ`--------------------------------------
// Flush to zero: force samples under the "maximum subnormal number"
// to be zero. Usually not needed in C++ because the architecture
// file take care of this, but can be useful in JavaScript for instance.
//
// #### Usage
//
// ```
// _ : FTZ : _
// ```
//
// Where:
//
// * `x`: input signal to flush if its magnitude is subnormal
//
// #### Test
// ```
// ma = library("maths.lib");
// FTZ_test = ((ma.MIN * 0.5) : ma.FTZ), ((ma.MIN * 1e307) : ma.FTZ);
// ```
//
// #### References
//
// * <http://docs.oracle.com/cd/E19957-01/806-3568/ncg_math.html>
//-------------------------------------------------------------------------
FTZ(x) = x * (abs(x) > MIN);


//----------------------------------`(ma.)copysign`---------------------------------------
// Changes the sign of x (first input) to that of y (second input).
//
// #### Usage
//
// ```
// _,_ : copysign : _
// ```
//
// Where:
//
// * `x`: value whose magnitude is preserved
// * `y`: value providing the sign
//
// #### Test
// ```
// ma = library("maths.lib");
// copysign_test = (-1.0, 2.0) : ma.copysign;
// ```
//-------------------------------------------------------------------------
copysign = ffunction(float copysignf|copysign|copysignl (float, float), <math.h>,"");


//----------------------------------`(ma.)neg`---------------------------------------
// Invert the sign (-x) of a signal.
//
```

```
// #### Usage
//
// ```
// _ : neg : _
// ```
//
// Where:
//
// * `x`: value to negate
//
// #### Test
// ```
// ma = library("maths.lib");
// neg_test = 3.5 : ma.neg;
// ```
//-----------------------------------------------------------------------
neg(x) = -x;


//---------------------------------`(ma.)not`-------------------------------------
// Bitwise `not` implemented with [xor](https://faustdoc.grame.fr/manual/syntax/#xor-primitive) as `not(x) = x xor -1;`.
// So working regardless of the size of the integer, assuming negative numbers in two's complement.
//
// #### Usage
//
// ```
// _ : not : _
// ```
//
// Where:
//
// * `x`: integer input value
//
// #### Test
// ```
// ma = library("maths.lib");
// not_test = 5 : ma.not;
// ```
//-----------------------------------------------------------------------
not(x) = x xor -1;


//-------`(ma.)sub(x,y)`------------------
// Subtract `x` and `y`.
//
// #### Usage
//
// ```
// _,_ : sub : _
// ```
//
// Where:
//
// * `x`: first operand
// * `y`: second operand
//
// #### Test
// ```
// ma = library("maths.lib");
// sub_test = (3, 10) : ma.sub;
// ```
//-----------------------------
sub(x,y) = y-x;


//---------------------------------`(ma.)inv`-----------------------------------------
```

```
// Compute the inverse (1/x) of the input signal.
//
// #### Usage
//
// ```
// _ : inv : _
// ```
//
// Where:
//
// * `x`: denominator input (non-zero)
//
// #### Test
// ```
// ma = library("maths.lib");
// inv_test = 4.0 : ma.inv;
// ```
//-------------------------------------------------------------------------
inv(x) = 1/x;


//----------------------------------`(ma.)cbrt`-------------------------------------
// Computes the cube root of of the input signal.
//
// #### Usage
//
// ```
// _ : cbrt : _
// ```
//
// Where:
//
// * `x`: value whose cube root is computed
//
// #### Test
// ```
// ma = library("maths.lib");
// cbrt_test = 8.0 : ma.cbrt;
// ```
//-------------------------------------------------------------------------
cbrt = ffunction(float cbrtf|cbrt|cbrtl (float), <math.h>,"");


//----------------------------------`(ma.)hypot`-------------------------------------
// Computes the euclidian distance of the two input signals
// sqrt(x*x+y*y) without undue overflow or underflow.
//
// #### Usage
//
// ```
// _,_ : hypot : _
// ```
//
// Where:
//
// * `x`: first operand
// * `y`: second operand
//
// #### Test
// ```
// ma = library("maths.lib");
// hypot_test = (3.0, 4.0) : ma.hypot;
// ```
//-------------------------------------------------------------------------
hypot = ffunction(float hypotf|hypot|hypotl (float, float), <math.h>,"");


//----------------------------------`(ma.)ldexp`-------------------------------------
```

```
// Takes two input signals: x and n, and multiplies x by 2 to the power n.
//
// #### Usage
//
// ```
// _,_ : ldexp : _
// ```
//
// Where:
//
// * `x`: significand input
// * `n`: exponent (integer) input
//
// #### Test
// ```
// ma = library("maths.lib");
// ldexp_test = (1.5, 3) : ma.ldexp;
// ```
//-------------------------------------------------------------------------
ldexp = ffunction(float ldexpf|ldexp|ldexpl (float, int), <math.h>,"");


//----------------------------------`(ma.)scalb`-----------------------------------
// Takes two input signals: x and n, and multiplies x by 2 to the power n.
//
// #### Usage
//
// ```
// _,_ : scalb : _
// ```
//
// Where:
//
// * `x`: significand input
// * `n`: exponent (integer) input
//
// #### Test
// ```
// ma = library("maths.lib");
// scalb_test = (2.0, -1) : ma.scalb;
// ```
//-------------------------------------------------------------------------
scalb = ffunction(float scalbnf|scalbn|scalbnl (float, int), <math.h>,"");


//----------------------------------`(ma.)log1p`-----------------------------------
// Computes log(1 + x) without undue loss of accuracy when x is nearly zero.
//
// #### Usage
//
// ```
// _ : log1p : _
// ```
//
// Where:
//
// * `x`: offset used in `log(1 + x)` (must be greater than -1)
//
// #### Test
// ```
// ma = library("maths.lib");
// log1p_test = 0.5 : ma.log1p;
// ```
//-------------------------------------------------------------------------
log1p = ffunction(float log1pf|log1p|log1pl (float), <math.h>,"");


//----------------------------------`(ma.)logb`-----------------------------------
```

```
// Return exponent of the input signal as a floating-point number.
//
// #### Usage
//
// ```
// _ : logb : _
// ```
//
// Where:
//
// * `x`: positive value whose exponent part is returned
//
// #### Test
// ```
// ma = library("maths.lib");
// logb_test = 8.0 : ma.logb;
// ```
//-------------------------------------------------------------------------
logb = ffunction(float logbf|logb|logbl (float), <math.h>,"");


//---------------------------------`(ma.)ilogb`-----------------------------------
// Return exponent of the input signal as an integer number.
//
// #### Usage
//
// ```
// _ : ilogb : _
// ```
//
// Where:
//
// * `x`: positive value whose exponent part is returned
//
// #### Test
// ```
// ma = library("maths.lib");
// ilogb_test = 8.0 : ma.ilogb;
// ```
//-------------------------------------------------------------------------
ilogb = ffunction(int ilogbf|ilogb|ilogbl (float), <math.h>,"");


//---------------------------------`(ma.)log2`-----------------------------------
// Returns the base 2 logarithm of x.
//
// #### Usage
//
// ```
// _ : log2 : _
// ```
//
// Where:
//
// * `x`: positive value whose base-2 logarithm is computed
//
// #### Test
// ```
// ma = library("maths.lib");
// log2_test = 8.0 : ma.log2;
// ```
//-------------------------------------------------------------------------
log2(x) = log(x)/log(2.0);


//---------------------------------`(ma.)expm1`-----------------------------------
// Return exponent of the input signal minus 1 with better precision.
//
```

```
633 // #### Usage
634 //
635 // ```
636 // _ : expm1 : _
637 // ```
638 //
639 // Where:
640 //
641 // * `x`: input value used for the `exp(x) - 1` computation
642 //
643 // #### Test
644 // ```
645 // ma = library("maths.lib");
646 // expm1_test = 0.5 : ma.expm1;
647 // ```
648 //-------------------------------------------------------------------------
649 expm1 = ffunction(float expm1f|expm1|expm1l (float), <math.h>,"");
650
651
652 //---------------------------------`(ma.)acosh`-----------------------------------
653 // Computes the principle value of the inverse hyperbolic cosine
654 // of the input signal.
655 //
656 // #### Usage
657 //
658 // ```
659 // _ : acosh : _
660 // ```
661 //
662 // Where:
663 //
664 // * `x`: input value (greater than or equal to 1)
665 //
666 // #### Test
667 // ```
668 // ma = library("maths.lib");
669 // acosh_test = 1.5 : ma.acosh;
670 // ```
671 //-------------------------------------------------------------------------
672 acosh = ffunction(float acoshf|acosh|acoshl (float), <math.h>, "");
673
674
675 //--------------------------------`(ma.)asinh`-----------------------------------
676 // Computes the inverse hyperbolic sine of the input signal.
677 //
678 // #### Usage
679 //
680 // ```
681 // _ : asinh : _
682 // ```
683 //
684 // Where:
685 //
686 // * `x`: input value
687 //
688 // #### Test
689 // ```
690 // ma = library("maths.lib");
691 // asinh_test = 0.5 : ma.asinh;
692 // ```
693 //-------------------------------------------------------------------------
694 asinh = ffunction(float asinhf|asinh|asinhl (float), <math.h>, "");
695
696
697 //--------------------------------`(ma.)atanh`-----------------------------------
698 // Computes the inverse hyperbolic tangent of the input signal.
699 //
700 // #### Usage
```

```
//
// ```
// _ : atanh : _
// ```
//
// Where:
//
// * `x`: input value in (-1, 1)
//
// #### Test
// ```
// ma = library("maths.lib");
// atanh_test = 0.5 : ma.atanh;
// ```
//-------------------------------------------------------------------------------
atanh = ffunction(float atanhf|atanh|atanhl (float), <math.h>, "");


//----------------------------------`(ma.)sinh`-------------------------------------
// Computes the hyperbolic sine of the input signal.
//
// #### Usage
//
// ```
// _ : sinh : _
// ```
//
// Where:
//
// * `x`: input value
//
// #### Test
// ```
// ma = library("maths.lib");
// sinh_test = 0.5 : ma.sinh;
// ```
//-------------------------------------------------------------------------------
sinh = ffunction(float sinhf|sinh|sinhl (float), <math.h>, "");


//----------------------------------`(ma.)cosh`-------------------------------------
// Computes the hyperbolic cosine of the input signal.
//
// #### Usage
//
// ```
// _ : cosh : _
// ```
//
// Where:
//
// * `x`: input value
//
// #### Test
// ```
// ma = library("maths.lib");
// cosh_test = 0.5 : ma.cosh;
// ```
//-------------------------------------------------------------------------------
cosh = ffunction(float coshf|cosh|coshl (float), <math.h>, "");


//----------------------------------`(ma.)tanh`-------------------------------------
// Computes the hyperbolic tangent of the input signal.
//
// #### Usage
//
// ```
```

```
// _ : tanh : _
// ```
//
// Where:
//
// * `x`: input value
//
// #### Test
// ```
// ma = library("maths.lib");
// tanh_test = 0.5 : ma.tanh;
// ```
//-------------------------------------------------------------------------------
tanh = ffunction(float tanhf|tanh|tanhl (float), <math.h>,"");


//----------------------------------`(ma.)erf`-----------------------------------
// Computes the error function of the input signal.
//
// #### Usage
//
// ```
// _ : erf : _
// ```
//
// Where:
//
// * `x`: input value
//
// #### Test
// ```
// ma = library("maths.lib");
// erf_test = 0.5 : ma.erf;
// ```
//-------------------------------------------------------------------------------
erf = ffunction(float erff|erf|erfl(float), <math.h>,"");


//----------------------------------`(ma.)erfc`----------------------------------
// Computes the complementary error function of the input signal.
//
// #### Usage
//
// ```
// _ : erfc : _
// ```
//
// Where:
//
// * `x`: input value
//
// #### Test
// ```
// ma = library("maths.lib");
// erfc_test = 0.5 : ma.erfc;
// ```
//-------------------------------------------------------------------------------
erfc = ffunction(float erfcf|erfc|erfcl(float), <math.h>,"");


//----------------------------------`(ma.)gamma`---------------------------------
// Computes the gamma function of the input signal.
//
// #### Usage
//
// ```
// _ : gamma : _
// ```
```

```
//
// Where:
//
// * `x`: positive input value
//
// #### Test
// ```
// ma = library("maths.lib");
// gamma_test = 3.0 : ma.gamma;
// ```
//-------------------------------------------------------------------------
gamma = ffunction(float tgammaf|tgamma|tgammal(float), <math.h>,"");


//----------------------------------`(ma.)lgamma`-----------------------------------
// Calculates the natural logorithm of the absolute value of
// the gamma function of the input signal.
//
// #### Usage
//
// ```
// _ : lgamma : _
// ```
//
// Where:
//
// * `x`: positive input value
//
// #### Test
// ```
// ma = library("maths.lib");
// lgamma_test = 3.0 : ma.lgamma;
// ```
//-------------------------------------------------------------------------
lgamma = ffunction(float lgammaf|lgamma|lgammal(float), <math.h>,"");


//----------------------------------`(ma.)J0`-----------------------------------
// Computes the Bessel function of the first kind of order 0
// of the input signal.
//
// #### Usage
//
// ```
// _ : J0 : _
// ```
//
// Where:
//
// * `x`: input value
//
// #### Test
// ```
// ma = library("maths.lib");
// J0_test = 1.0 : ma.J0;
// ```
//-------------------------------------------------------------------------
J0 = ffunction(float j0(float), <math.h>,"");


//----------------------------------`(ma.)J1`-----------------------------------
// Computes the Bessel function of the first kind of order 1
// of the input signal.
//
// #### Usage
//
// ```
// _ : J1 : _
```

```
// ```
//
// Where:
//
// * `x`: input value
//
// #### Test
// ```
// ma = library("maths.lib");
// J1_test = 1.0 : ma.J1;
// ```
//-------------------------------------------------------------------------
J1 = ffunction(float j1(float), <math.h>,"");


//----------------------------------`(ma.)Jn`---------------------------------------
// Computes the Bessel function of the first kind of order n
// (first input signal) of the second input signal.
//
// #### Usage
//
// ```
// _,_ : Jn : _
// ```
//
// Where:
//
// * `n`: integer order
// * `x`: input value
//
// #### Test
// ```
// ma = library("maths.lib");
// Jn_test = (2, 1.0) : ma.Jn;
// ```
//-------------------------------------------------------------------------
Jn = ffunction(float jn(int, float), <math.h>,"");


//----------------------------------`(ma.)Y0`---------------------------------------
// Computes the linearly independent Bessel function of the second kind
// of order 0 of the input signal.
//
// #### Usage
//
// ```
// _ : Y0 : _
// ```
//
// Where:
//
// * `x`: positive input value
//
// #### Test
// ```
// ma = library("maths.lib");
// Y0_test = 1.0 : ma.Y0;
// ```
//-------------------------------------------------------------------------
Y0 = ffunction(float y0(float), <math.h>,"");


//----------------------------------`(ma.)Y1`---------------------------------------
// Computes the linearly independent Bessel function of the second kind
// of order 1 of the input signal.
//
// #### Usage
//
```

```
// ```
// _ : Y0 : _
// ```
//
// Where:
//
// * `x`: positive input value
//
// #### Test
// ```
// ma = library("maths.lib");
// Y1_test = 1.0 : ma.Y1;
// ```
//-------------------------------------------------------------------------
Y1 = ffunction(float y1(float), <math.h>,"");


//----------------------------------`(ma.)Yn`--------------------------------------
// Computes the linearly independent Bessel function of the second kind
// of order n (first input signal) of the second input signal.
//
// #### Usage
//
// ```
// _,_ : Yn : _
// ```
//
// Where:
//
// * `n`: integer order
// * `x`: positive input value
//
// #### Test
// ```
// ma = library("maths.lib");
// Yn_test = (2, 1.0) : ma.Yn;
// ```
//-------------------------------------------------------------------------
Yn = ffunction(float yn(int, float), <math.h>,"");


//----------------------------`(ma.)fabs`, `(ma.)fmax`, `(ma.)fmin
    `--------------------------
// Just for compatibility...
//
// ```
// fabs = abs
// fmax = max
// fmin = min
// ```
//-------------------------------------------------------------------------
fabs = abs;
fmax = max;
fmin = min;

//-------------------------------`(ma.)np2`--------------------------------------
// Gives the next power of 2 of x.
//
// #### Usage
//
// ```
// np2(n) : _
// ```
//
// Where:
//
// * `n`: an integer
//
```

```
// #### Test
// ```
// ma = library("maths.lib");
// np2_test = 5 : ma.np2;
// ```
//-----------------------------------------------------------------------------
np2 = -(1) <: >>(1)|_ <: >>(2)|_ <: >>(4)|_ <: >>(8)|_ <: >>(16)|_ : +(1);


//------------------------------`(ma.)frac`--------------------------------------
// Gives the fractional part of n.
//
// #### Usage
//
// ```
// frac(n) : _
// ```
//
// Where:
//
// * `n`: a decimal number
//
// #### Test
// ```
// ma = library("maths.lib");
// frac_test = 3.75 : ma.frac;
// ```
//-----------------------------------------------------------------------------
frac(n) = n - floor(n);
decimal = frac;

// NOTE: decimal does the same thing as frac but using floor instead. JOS uses frac a lot
// in filters.lib so we decided to keep that one... decimal is declared though for
// backward compatibility.
// decimal(n) = n - floor(n);


//--------------------------------`(ma.)modulo`--------------------------------------
// Modulus operation using the `(x%y+y)%y` formula to ensures the result is always non-
     negative, even if `x` is negative.
//
// #### Usage
//
// ```
// modulo(x,y) : _
// ```
//
// Where:
//
// * `x`: the numerator
// * `y`: the denominator
//
// #### Test
// ```
// ma = library("maths.lib");
// modulo_test = (-3, 4) : ma.modulo;
// ```
//-----------------------------------------------------------------------------
modulo(x,y) = (x % y + y) % y;


//---------------`(ma.)isnan`----------------
// Return non-zero if x is a NaN.
//
// #### Usage
//
// ```
// isnan(x)
```

```
// _ : isnan : _
// ```
//
// Where:
//
// * `x`: signal to analyse
//
// #### Test
// ```
// ma = library("maths.lib");
// os = library("oscillators.lib");
// isnan_test = (os.osc(1) - 2.0) : sqrt : ma.isnan;
// ```
//-----------------------------------------
isnan = ffunction(int isnanf|isnan|isnanl (float),<math.h>,"");


//---------------`(ma.)isinf`----------------
// Return non-zero if x is a positive or negative infinity.
//
// #### Usage
//
// ```
// isinf(x)
// _ : isinf : _
// ```
//
// Where:
//
// * `x`: signal to analyse
//
// #### Test
// ```
// ma = library("maths.lib");
// os = library("oscillators.lib");
// isinf_test = (os.impulse - os.impulse) : log : ma.isinf;
// ```
//-----------------------------------------
isinf = ffunction(int isinff|isinf|isinfl (float),<math.h>,"");

nextafter = ffunction(float nextafter(float, float),<math.h>,"");


//---------------------------`(ma.)chebychev`-------------------------------
// Chebychev transformation of order N.
//
// #### Usage
//
// ```
// _ : chebychev(N) : _
// ```
//
// Where:
//
// * `N`: the order of the polynomial, a constant numerical expression
//
// #### Semantics
//
// ```
// T[0](x) = 1,
// T[1](x) = x,
// T[n](x) = 2x*T[n-1](x) - T[n-2](x)
// ```
//
// #### Test
// ```
// ma = library("maths.lib");
// chebychev_test = 0.5 : ma.chebychev(3);
```

```
// ```
//
// #### References
//
// * <http://en.wikipedia.org/wiki/Chebyshev_polynomial>
//--------------------------------------------------------------------------
chebychev(0,x) = 1;
chebychev(1,x) = x;
chebychev(n,x) = 2*x*chebychev(n-1, x) - chebychev(n-2, x);


//------------------------`(ma.)chebychevpoly`-------------------------------
// Linear combination of the first Chebyshev polynomials.
//
// #### Usage
//
// ```
// _ : chebychevpoly((c0,c1,...,cn)) : _
// ```
//
// Where:
//
// * `cn`: the different Chebychevs polynomials such that:
// chebychevpoly((c0,c1,...,cn)) = Sum of chebychev(i)*ci
//
// #### Test
// ```
// ma = library("maths.lib");
// chebychevpoly_test = 0.5 : ma.chebychevpoly((1, 0, 1));
// ```
//
// #### References
//
// * <http://www.csounds.com/manual/html/chebyshevpoly.html>
//--------------------------------------------------------------------------
chebychevpoly(lcoef) = _ <: L(0,lcoef) :> _
    with {
        L(n,(c,cs)) = chebychev(n)*c, L(n+1,cs);
        L(n,c)    = chebychev(n)*c;
    };


//------------------`(ma.)diffn`----------------------------
// Negated first-order difference.
//
// #### Usage
//
// ```
// _ : diffn : _
// ```
//
// Where:
//
// * `x`: input signal
//
// #### Test
// ```
// ma = library("maths.lib");
// os = library("oscillators.lib");
// diffn_test = os.osc(440) : ma.diffn;
// ```
//-----------------------------------------------------
diffn(x) = x' - x; // negated first-order difference


//------------------`(ma.)signum`-----------------------------
// The signum function signum(x) is defined as
// -1 for x<0, 0 for x==0, and 1 for x>0.
```

```
//
// #### Usage
//
// ```
// _ : signum : _
// ```
//
// Where:
//
// * `x`: input value
//
// #### Test
// ```
// ma = library("maths.lib");
// signum_test = (-5.0) : ma.signum;
// ```
//-------------------------------------------------------
signum(x) = (x>0)-(x<0);


//------------------`(ma.)nextpow2`----------------------------
// The nextpow2(x) returns the lowest integer m such that
// 2^m >= x.
//
// #### Usage
//
// ```
// 2^nextpow2(n) : _
// ```
// Useful for allocating delay lines, e.g.,
// ```
// delay(2^nextpow2(maxDelayNeeded), currentDelay);
// ```
//
// Where:
//
// * `n`: positive value whose next power-of-two exponent is computed
//
// #### Test
// ```
// ma = library("maths.lib");
// nextpow2_test = 10.0 : ma.nextpow2;
// ```
//-------------------------------------------------------
nextpow2(x) = ceil(log(x)/log(2.0));


//--------------------`(ma.)zc`-----------------------------------------------
// Indicator function for zero-crossing: it returns 1 if a zero-crossing
// occurs, 0 otherwise.
//
// #### Usage
//
// ```
// _ : zc : _
// ```
//
// Where:
//
// * `x`: input signal to monitor for zero crossings
//
// #### Test
// ```
// ma = library("maths.lib");
// os = library("oscillators.lib");
// zc_test = os.osc(440) : ma.zc;
// ```
//-------------------------------------------------------------------------
```

```
zc = ba.tAndH(!=(0)) <: *(mem) < 0;

//-------------------------`(ma.)unwrap` ---------------------------------------
// Unwrap the input signal so that successive output values never differ
// by more than `pi`, switching to a 2*pi-complementary value when needed.
//
// #### Usage
//
// ```
// _ : unwrap(pi) : _
// ```
//
// Where:
//
// * `pi`: maximum discontinuity between the output values (typically `ma.PI`)
//
// #### Test
// ```
// ma = library("maths.lib");
// os = library("oscillators.lib");
// unwrap_test = os.oscrc(100) : ma.unwrap(ma.PI);
// ```
//
// #### Example test program
//
// ```
// process = 0 - os.oscrc(1000)    // the true phase is either -PI or +PI
//         : an.resonator(1,1000) : !,_ // oscillates between -PI and +PI
//         : ma.unwrap(ma.PI);      // oscillates near +PI
// ```
//
//-------------------------------------------------------------------------
unwrap(pi,x) = (_ <: +(dz)) ~ _ with {
    dz(z) = wrap(x - z, 2*pi) <: ba.ifNc(1, >(pi),-(2*pi));
    wrap(x,d) = x - d * floor(x / d);
};

//--------------------`(ma.)primes`-----------------------------------------------
// Return the n-th prime using a waveform primitive. Note that primes(0) is 2,
// primes(1) is 3, and so on. The waveform is length 2048, so the largest
// precomputed prime is primes(2047) which is 17863.
//
// #### Usage
//
// ```
// _ : primes : _
// ```
//
// Where:
//
// * `x`: index of the prime number sequence (0-based).
//
// #### Test
// ```
// ma = library("maths.lib");
// primes_test = 10 : ma.primes;
// ```
//-------------------------------------------------------------------------
primes(x) = rdtable(waveform{
2,3,5,7,11,13,17,19,23,29,
31,37,41,43,47,53,59,61,67,71,
73,79,83,89,97,101,103,107,109,113,
127,131,137,139,149,151,157,163,167,173,
179,181,191,193,197,199,211,223,227,229,
233,239,241,251,257,263,269,271,277,281,
283,293,307,311,313,317,331,337,347,349,
353,359,367,373,379,383,389,397,401,409,
419,421,431,433,439,443,449,457,461,463,
```

```
467,479,487,491,499,503,509,521,523,541,
547,557,563,569,571,577,587,593,599,601,
607,613,617,619,631,641,643,647,653,659,
661,673,677,683,691,701,709,719,727,733,
739,743,751,757,761,769,773,787,797,809,
811,821,823,827,829,839,853,857,859,863,
877,881,883,887,907,911,919,929,937,941,
947,953,967,971,977,983,991,997,1009,1013,
1019,1021,1031,1033,1039,1049,1051,1061,1063,1069,
1087,1091,1093,1097,1103,1109,1117,1123,1129,1151,
1153,1163,1171,1181,1187,1193,1201,1213,1217,1223,
1229,1231,1237,1249,1259,1277,1279,1283,1289,1291,
1297,1301,1303,1307,1319,1321,1327,1361,1367,1373,
1381,1399,1409,1423,1427,1429,1433,1439,1447,1451,
1453,1459,1471,1481,1483,1487,1489,1493,1499,1511,
1523,1531,1543,1549,1553,1559,1567,1571,1579,1583,
1597,1601,1607,1609,1613,1619,1621,1627,1637,1657,
1663,1667,1669,1693,1697,1699,1709,1721,1723,1733,
1741,1747,1753,1759,1777,1783,1787,1789,1801,1811,
1823,1831,1847,1861,1867,1871,1873,1877,1879,1889,
1901,1907,1913,1931,1933,1949,1951,1973,1979,1987,
1993,1997,1999,2003,2011,2017,2027,2029,2039,2053,
2063,2069,2081,2083,2087,2089,2099,2111,2113,2129,
2131,2137,2141,2143,2153,2161,2179,2203,2207,2213,
2221,2237,2239,2243,2251,2267,2269,2273,2281,2287,
2293,2297,2309,2311,2333,2339,2341,2347,2351,2357,
2371,2377,2381,2383,2389,2393,2399,2411,2417,2423,
2437,2441,2447,2459,2467,2473,2477,2503,2521,2531,
2539,2543,2549,2551,2557,2579,2591,2593,2609,2617,
2621,2633,2647,2657,2659,2663,2671,2677,2683,2687,
2689,2693,2699,2707,2711,2713,2719,2729,2731,2741,
2749,2753,2767,2777,2789,2791,2797,2801,2803,2819,
2833,2837,2843,2851,2857,2861,2879,2887,2897,2903,
2909,2917,2927,2939,2953,2957,2963,2969,2971,2999,
3001,3011,3019,3023,3037,3041,3049,3061,3067,3079,
3083,3089,3109,3119,3121,3137,3163,3167,3169,3181,
3187,3191,3203,3209,3217,3221,3229,3251,3253,3257,
3259,3271,3299,3301,3307,3313,3319,3323,3329,3331,
3343,3347,3359,3361,3371,3373,3389,3391,3407,3413,
3433,3449,3457,3461,3463,3467,3469,3491,3499,3511,
3517,3527,3529,3533,3539,3541,3547,3557,3559,3571,
3581,3583,3593,3607,3613,3617,3623,3631,3637,3643,
3659,3671,3673,3677,3691,3697,3701,3709,3719,3727,
3733,3739,3761,3767,3769,3779,3793,3797,3803,3821,
3823,3833,3847,3851,3853,3863,3877,3881,3889,3907,
3911,3917,3919,3923,3929,3931,3943,3947,3967,3989,
4001,4003,4007,4013,4019,4021,4027,4049,4051,4057,
4073,4079,4091,4093,4099,4111,4127,4129,4133,4139,
4153,4157,4159,4177,4201,4211,4217,4219,4229,4231,
4241,4243,4253,4259,4261,4271,4273,4283,4289,4297,
4327,4337,4339,4349,4357,4363,4373,4391,4397,4409,
4421,4423,4441,4447,4451,4457,4463,4481,4483,4493,
4507,4513,4517,4519,4523,4547,4549,4561,4567,4583,
4591,4597,4603,4621,4637,4639,4643,4649,4651,4657,
4663,4673,4679,4691,4703,4721,4723,4729,4733,4751,
4759,4783,4787,4789,4793,4799,4801,4813,4817,4831,
4861,4871,4877,4889,4903,4909,4919,4931,4933,4937,
4943,4951,4957,4967,4969,4973,4987,4993,4999,5003,
5009,5011,5021,5023,5039,5051,5059,5077,5081,5087,
5099,5101,5107,5113,5119,5147,5153,5167,5171,5179,
5189,5197,5209,5227,5231,5233,5237,5261,5273,5279,
5281,5297,5303,5309,5323,5333,5347,5351,5381,5387,
5393,5399,5407,5413,5417,5419,5431,5437,5441,5443,
5449,5471,5477,5479,5483,5501,5503,5507,5519,5521,
5527,5531,5557,5563,5569,5573,5581,5591,5623,5639,
5641,5647,5651,5653,5657,5659,5669,5683,5689,5693,
5701,5711,5717,5737,5741,5743,5749,5779,5783,5791,
5801,5807,5813,5821,5827,5839,5843,5849,5851,5857,
```

```
5861,5867,5869,5879,5881,5897,5903,5923,5927,5939,
5953,5981,5987,6007,6011,6029,6037,6043,6047,6053,
6067,6073,6079,6089,6091,6101,6113,6121,6131,6133,
6143,6151,6163,6173,6197,6199,6203,6211,6217,6221,
6229,6247,6257,6263,6269,6271,6277,6287,6299,6301,
6311,6317,6323,6329,6337,6343,6353,6359,6361,6367,
6373,6379,6389,6397,6421,6427,6449,6451,6469,6473,
6481,6491,6521,6529,6547,6551,6553,6563,6569,6571,
6577,6581,6599,6607,6619,6637,6653,6659,6661,6673,
6679,6689,6691,6701,6703,6709,6719,6733,6737,6761,
6763,6779,6781,6791,6793,6803,6823,6827,6829,6833,
6841,6857,6863,6869,6871,6883,6899,6907,6911,6917,
6947,6949,6959,6961,6967,6971,6977,6983,6991,6997,
7001,7013,7019,7027,7039,7043,7057,7069,7079,7103,
7109,7121,7127,7129,7151,7159,7177,7187,7193,7207,
7211,7213,7219,7229,7237,7243,7247,7253,7283,7297,
7307,7309,7321,7331,7333,7349,7351,7369,7393,7411,
7417,7433,7451,7457,7459,7477,7481,7487,7489,7499,
7507,7517,7523,7529,7537,7541,7547,7549,7559,7561,
7573,7577,7583,7589,7591,7603,7607,7621,7639,7643,
7649,7669,7673,7681,7687,7691,7699,7703,7717,7723,
7727,7741,7753,7757,7759,7789,7793,7817,7823,7829,
7841,7853,7867,7873,7877,7879,7883,7901,7907,7919,
7927,7933,7937,7949,7951,7963,7993,8009,8011,8017,
8039,8053,8059,8069,8081,8087,8089,8093,8101,8111,
8117,8123,8147,8161,8167,8171,8179,8191,8209,8219,
8221,8231,8233,8237,8243,8263,8269,8273,8287,8291,
8293,8297,8311,8317,8329,8353,8363,8369,8377,8387,
8389,8419,8423,8429,8431,8443,8447,8461,8467,8501,
8513,8521,8527,8537,8539,8543,8563,8573,8581,8597,
8599,8609,8623,8627,8629,8641,8647,8663,8669,8677,
8681,8689,8693,8699,8707,8713,8719,8731,8737,8741,
8747,8753,8761,8779,8783,8803,8807,8819,8821,8831,
8837,8839,8849,8861,8863,8867,8887,8893,8923,8929,
8933,8941,8951,8963,8969,8971,8999,9001,9007,9011,
9013,9029,9041,9043,9049,9059,9067,9091,9103,9109,
9127,9133,9137,9151,9157,9161,9173,9181,9187,9199,
9203,9209,9221,9227,9239,9241,9257,9277,9281,9283,
9293,9311,9319,9323,9337,9341,9343,9349,9371,9377,
9391,9397,9403,9413,9419,9421,9431,9433,9437,9439,
9461,9463,9467,9473,9479,9491,9497,9511,9521,9533,
9539,9547,9551,9587,9601,9613,9619,9623,9629,9631,
9643,9649,9661,9677,9679,9689,9697,9719,9721,9733,
9739,9743,9749,9767,9769,9781,9787,9791,9803,9811,
9817,9829,9833,9839,9851,9857,9859,9871,9883,9887,
9901,9907,9923,9929,9931,9941,9949,9967,9973,10007,
10009,10037,10039,10061,10067,10069,10079,10091,10093,10099,
10103,10111,10133,10139,10141,10151,10159,10163,10169,10177,
10181,10193,10211,10223,10243,10247,10253,10259,10267,10271,
10273,10289,10301,10303,10313,10321,10331,10333,10337,10343,
10357,10369,10391,10399,10427,10429,10433,10453,10457,10459,
10463,10477,10487,10499,10501,10513,10529,10531,10559,10567,
10589,10597,10601,10607,10613,10627,10631,10639,10651,10657,
10663,10667,10687,10691,10709,10711,10723,10729,10733,10739,
10753,10771,10781,10789,10799,10831,10837,10847,10853,10859,
10861,10867,10883,10889,10891,10903,10909,10937,10939,10949,
10957,10973,10979,10987,10993,11003,11027,11047,11057,11059,
11069,11071,11083,11087,11093,11113,11117,11119,11131,11149,
11159,11161,11171,11173,11177,11197,11213,11239,11243,11251,
11257,11261,11273,11279,11287,11299,11311,11317,11321,11329,
11351,11353,11369,11383,11393,11399,11411,11423,11437,11443,
11447,11467,11471,11483,11489,11491,11497,11503,11519,11527,
11549,11551,11579,11587,11593,11597,11617,11621,11633,11657,
11677,11681,11689,11699,11701,11717,11719,11731,11743,11777,
11779,11783,11789,11801,11807,11813,11821,11827,11831,11833,
11839,11863,11867,11887,11897,11903,11909,11923,11927,11933,
11939,11941,11953,11959,11969,11971,11981,11987,12007,12011,
12037,12041,12043,12049,12071,12073,12097,12101,12107,12109,
```

```
12113,12119,12143,12149,12157,12161,12163,12197,12203,12211,
12227,12239,12241,12251,12253,12263,12269,12277,12281,12289,
12301,12323,12329,12343,12347,12373,12377,12379,12391,12401,
12409,12413,12421,12433,12437,12451,12457,12473,12479,12487,
12491,12497,12503,12511,12517,12527,12539,12541,12547,12553,
12569,12577,12583,12589,12601,12611,12613,12619,12637,12641,
12647,12653,12659,12671,12689,12697,12703,12713,12721,12739,
12743,12757,12763,12781,12791,12799,12809,12821,12823,12829,
12841,12853,12889,12893,12899,12907,12911,12917,12919,12923,
12941,12953,12959,12967,12973,12979,12983,13001,13003,13007,
13009,13033,13037,13043,13049,13063,13093,13099,13103,13109,
13121,13127,13147,13151,13159,13163,13171,13177,13183,13187,
13217,13219,13229,13241,13249,13259,13267,13291,13297,13309,
13313,13327,13331,13337,13339,13367,13381,13397,13399,13411,
13417,13421,13441,13451,13457,13463,13469,13477,13487,13499,
13513,13523,13537,13553,13567,13577,13591,13597,13613,13619,
13627,13633,13649,13669,13679,13681,13687,13691,13693,13697,
13709,13711,13721,13723,13729,13751,13757,13759,13763,13781,
13789,13799,13807,13829,13831,13841,13859,13873,13877,13879,
13883,13901,13903,13907,13913,13921,13931,13933,13963,13967,
13997,13999,14009,14011,14029,14033,14051,14057,14071,14081,
14083,14087,14107,14143,14149,14153,14159,14173,14177,14197,
14207,14221,14243,14249,14251,14281,14293,14303,14321,14323,
14327,14341,14347,14369,14387,14389,14401,14407,14411,14419,
14423,14431,14437,14447,14449,14461,14479,14489,14503,14519,
14533,14537,14543,14549,14551,14557,14561,14563,14591,14593,
14621,14627,14629,14633,14639,14653,14657,14669,14683,14699,
14713,14717,14723,14731,14737,14741,14747,14753,14759,14767,
14771,14779,14783,14797,14813,14821,14827,14831,14843,14851,
14867,14869,14879,14887,14891,14897,14923,14929,14939,14947,
14951,14957,14969,14983,15013,15017,15031,15053,15061,15073,
15077,15083,15091,15101,15107,15121,15131,15137,15139,15149,
15161,15173,15187,15193,15199,15217,15227,15233,15241,15259,
15263,15269,15271,15277,15287,15289,15299,15307,15313,15319,
15329,15331,15349,15359,15361,15373,15377,15383,15391,15401,
15413,15427,15439,15443,15451,15461,15467,15473,15493,15497,
15511,15527,15541,15551,15559,15569,15581,15583,15601,15607,
15619,15629,15641,15643,15647,15649,15661,15667,15671,15679,
15683,15727,15731,15733,15737,15739,15749,15761,15767,15773,
15787,15791,15797,15803,15809,15817,15823,15859,15877,15881,
15887,15889,15901,15907,15913,15919,15923,15937,15959,15971,
15973,15991,16001,16007,16033,16057,16061,16063,16067,16069,
16073,16087,16091,16097,16103,16111,16127,16139,16141,16183,
16187,16189,16193,16217,16223,16229,16231,16249,16253,16267,
16273,16301,16319,16333,16339,16349,16361,16363,16369,16381,
16411,16417,16421,16427,16433,16447,16451,16453,16477,16481,
16487,16493,16519,16529,16547,16553,16561,16567,16573,16603,
16607,16619,16631,16633,16649,16651,16657,16661,16673,16691,
16693,16699,16703,16729,16741,16747,16759,16763,16787,16811,
16823,16829,16831,16843,16871,16879,16883,16889,16901,16903,
16921,16927,16931,16937,16943,16963,16979,16981,16987,16993,
17011,17021,17027,17029,17033,17041,17047,17053,17077,17093,
17099,17107,17117,17123,17137,17159,17167,17183,17189,17191,
17203,17207,17209,17231,17239,17257,17291,17293,17299,17317,
17321,17327,17333,17341,17351,17359,17377,17383,17387,17389,
17393,17401,17417,17419,17431,17443,17449,17467,17471,17477,
17483,17489,17491,17497,17509,17519,17539,17551,17569,17573,
17579,17581,17597,17599,17609,17623,17627,17657,17659,17669,
17681,17683,17707,17713,17729,17737,17747,17749,17761,17783,
17789,17791,17807,17827,17837,17839,17851,17863
}, x);
```

Listing 7: `platform.lib`

```
//################################### platform.lib ########################################
// A library to handle platform specific code in Faust. Its official prefix is `pl`.
//
// #### References
//
// * <https://github.com/grame-cncm/faustlibraries/blob/master/platform.lib>

//#############################################################################################
// It can be reimplemented to globally change the SR and the tablesize definitions


/************************************************************************
*************************************************************************
FAUST library file
Copyright (C) 2020 GRAME, Centre National de Creation Musicale
--------------------------------------------------------------------
This program is free software; you can redistribute it and/or modify
it under the terms of the GNU Lesser General Public License as
published by the Free Software Foundation; either version 2.1 of the
License, or (at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public
License along with the GNU C Library; if not, write to the Free
Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
02111-1307 USA.

EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
larger FAUST program which directly or indirectly imports this library
file and still distribute the compiled code generated by the FAUST
compiler, or a modified version of this compiled code, under your own
copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
grants you the right to freely choose the license for the resulting
compiled code. In particular the resulting compiled code has no obligation
to be LGPL or GPL. For example you are free to choose a commercial or
closed source license or any other license if you decide so.
*************************************************************************
*************************************************************************/

declare name "Generic Platform Library";
declare version "1.3.0";

//----------------------------------`(pl.)SR`-----------------------------------
// Current sampling rate (between 1 and 192000Hz). Constant during
// program execution. Setting this value to a constant will allow the
// compiler to optimize the code by computing constant expressions at
// compile time, and can be valuable for performance, especially on
// embedded systems.
//------------------------------------------------------------------------------
SR = min(192000.0, max(1.0, fconstant(int fSamplingFreq, <math.h>)));

//----------------------------------`(pl.)BS`-----------------------------------
// Current block-size (between 1 and 16384 frames). Can change during the execution.
//------------------------------------------------------------------------------
BS = min(16384.0, max(1.0, fvariable(int count, <math.h>)));

//----------------------------------`(pl.)tablesize`----------------------------
// Oscillator table size. This value is used to define the size of the
// table used by the oscillators. It is usually a power of 2 and can be lowered
// to save memory. The default value is 65536.
//------------------------------------------------------------------------------
tablesize = 1 << 16;
```